

A Domain Generalization Perspective on Listwise Context Modeling

Lin Zhu, Yihong Chen, Bowen He

Ctrip Travel Network Technology Co., Limited.
{zhulb, yihongchen, bwhe}@ctrip.com

Abstract

As one of the most popular techniques for solving the ranking problem in information retrieval, Learning-to-rank (LETOR) has received a lot of attention both in academia and industry due to its importance in a wide variety of data mining applications. However, most of existing LETOR approaches choose to learn a single global ranking function to handle all queries, and ignore the substantial differences that exist between queries. In this paper, we propose a domain generalization strategy to tackle this problem. We propose Query-Invariant Listwise Context Modeling (QILCM), a novel neural architecture which eliminates the detrimental influence of inter-query variability by learning *query-invariant* latent representations, such that the ranking system could generalize better to unseen queries. We evaluate our techniques on benchmark datasets, demonstrating that QILCM outperforms previous state-of-the-art approaches by a substantial margin.

Introduction

As an important learning paradigm for tackling the ranking problem in information retrieval, learning-to-rank (LETOR) has received a lot of attention both in academia and industry due to its importance in a wide variety of applications, such as document retrieval (?; ?), recommendation systems (?; ?), and E-commerce search (?; ?).

The typical assumption behind LETOR methods is that for any given query, the relevance score of a candidate item is a parameterized *global* function of a set of features that describe the query, the item, and their interactions (?; ?). The parameters of this function can be determined by fitting it to a training set that consists of query-item pairs and the associated relevance judgments. Once learned, the function is applied to handle any future query to rank the candidate items with respect to the query. However, queries may vary significantly in the underlying users' intentions (?), in the distributions of relevant items (?), or in the features that are relevant to item ranking (?), and it is difficult to learn a single model that could encompass all these diversities. Moreover, given the substantial variations between queries, it is quite possible that a unseen test query have certain characteristics that are rarely encountered in the train-

ing set, leading to degradation in generalization ability of the ranking function (?).

So far, a number of approaches have been proposed to tackle the aforementioned problem. In particular, it has been demonstrated that the performance of a global ranking model can be greatly improved by incorporating the *local ranking context* information (?), which is generally extracted from the input data via feature engineering efforts or novel model architectures (?; ?; ?; ?). Although some promising results have been shown, these approaches still do not explicitly filter out the detrimental variations of queries, which may increase the risk of overfitting and hurt the performance.

At another extreme, we could train a ranking model for every query independently (?; ?). While this strategy may avoid the negative influence of the heterogeneity of queries, it does not allow information to be shared and transferred between queries, which makes generalization to unseen queries even harder.

A better solution may lie in the middle of these two extremes: we still learn a common ranking model for all queries, and yet treat every query as a separate *domain*. All of the queries (domains) share the same learning task and the same input feature space, but have different distributions. In recognition of the variations between queries (domains), we would like the model to acquire knowledge from 'source' queries (domains) that are available as the training set, and adapt the learned knowledge to unknown 'target' queries (domains).

In this formulation, we notice that the LETOR problem is equivalent to a *domain generalization (DG)* problem (?), where the objective is precisely to infer a learning system that can take as input a set of training domains and will output a model that can achieve high accuracy for new domains. To achieve this goal, many existing DG methods propose to learn *domain-invariant* representations that are expected to remove the negative effects of distributional changes across domains (?; ?).

On the basis of the above considerations, in this paper we propose to solve the query heterogeneity problem via DG techniques. The main contributions of our work are summarized as follows:

- We introduce a DG formulation of the LETOR problem and discuss why a DG perspective is justified in this set-

ting;

- We propose Query-Invariant Listwise Context Model (QILCM), a novel neural architecture for DG in this LETOR context;
- We perform comprehensive evaluations on three benchmark datasets, demonstrating that QILCM outperforms previous state-of-the-art approaches by a substantial margin.

Related Work

LETOR methods

According to how training losses are formulated, existing algorithms for LETOR can be broadly categorized into three categories: (1) The pointwise approaches (??), which reformulates LETOR as a classification or regression problem on single items; (2) The pairwise approaches (??), which formulate LETOR as a classification problem on the item pairs; (3) The listwise approaches (??) which directly optimize the ranking metrics. As mentioned in the introduction section, queries could vary greatly in various aspects, and exhibit different ‘standards’ for ranking (?). As a result, listwise and pairwise LETOR methods, which mainly focus on modeling the relative (instead of absolute) preferences between items for a query, generally perform better than pointwise approaches in practice (??; ??; ?). Our proposed DG perspective for LETOR could be considered as a continuation of these methods, since it more explicitly enforces the model to filter out the ‘global’ differences between queries, and focus on the ‘relative’ differences between items for each given query. Meanwhile, most of existing listwise and pairwise LETOR methods still try to learn a global ranking function that assesses the relevance of each item individually. In contrast, listwise context modeling methods studied in this paper treat the candidate items as a whole, and can better model their mutual influences (?).

Neural Ranking Models

Deep neural network methods have been applied to numerous ranking applications, such as recommendation systems (?), ad-hoc retrieval (??; ?), and context-aware ranking (?). Similar to the traditional LETOR methods discussed above, these models also score each item independently and neglect the negative influence of distributional differences between queries. This limitation is clearly demonstrated in (?), which shows that these models could overfit to the domains from which the training data is sampled, and generalize poorly to domains not encountered during training. To handle this problem, Cohen et al. also propose learning of domain invariant representations. However, they assume that the queries have already been categorized into a few broad domains and focus on avoidance of overfitting to these predetermined domains, while our method do not rely on such additional supervisory information, and instead focus directly on tackling the finer-grained variations between queries.

Context Aware Ranking

A significant amount of research has focused on leveraging contextual data to improve ranking. In particular, certain types of contextual information have been explored in depth, such as temporal dynamics of user behaviors (??; ??) and the geological location data (??; ?).

On the other hand, modeling of the local ranking context formed by candidate items is a less well studied problem, and state-of-the-art approaches are generally based on the Recurrent Neural Network (RNN) models (??; ??; ?). For example, (?) reformulate the ranking task as a RNN-based sequence generation problem and use the beam search algorithm to generate the ranked item list, while Deep Choice Model (DCM) (?) and Deep Listwise Context Model (DLCM) (?) adopt RNNs to generate a context encoding that summarizes the candidate items, which is then adopted to re-query the candidate items to generate the ranking scores. Although some promising results have been shown, a potential limitation of these methods is that RNNs are mainly designed for modeling sequential data, and yet the data encountered in ranking problems cannot always be organized as a natural and unique sequence. For example, the candidate items may be selected from a large repository using some initial retrieval methods (??; ?), and are essentially orderless. For such problems, although one can still order the candidate items according to certain pre-specified rules, and in principle universal approximators such as RNNs should be robust to choices of such pre-ordering, previous studies nevertheless show that the input order enforces a implicit prior on how RNNs should encode the given data, and could have strong impact on the experimental performance (?). Meanwhile, by adopting additional components such as positional encoding (?), models without recurrent structures can also incorporate useful order information to achieve good performance (??; ??; ?).

In addition, it is still necessary for these approaches to learn from the data to filter out the detrimental variations of queries, while preserving the necessary information for the ranking task. Better performance may be obtained by relieving such a learning burden using certain dedicated architectures that explicitly encode robustness to inter-query variations, such as the DG models reviewed later.

Domain Generalization (DG)

So far, a large number of formulations have been proposed to solve the DG problem, we refer the interested readers to (??; ?) for up-to-date reviews of related techniques. Our method is directly inspired by the DG approaches which learn a high-level data representation that simultaneously minimizes domain dissimilarity and preserves the functional relationship with the learning target. To the best of our knowledge, this is the first time that such techniques are applied to model the local ranking context formed by candidate items.

Methodology

Problem Formulation

Let $\mathcal{Q}$ be the set of possible queries and $\mathcal{D}$ be the set of possible items. Assume we are given a set of queries $\mathcal{T} \subseteq \mathcal{Q}$ as the training data, where each query $q \in \mathcal{T}$ is provided with a set of n_q candidate items $\{d_{q,1}, d_{q,2}, \dots, d_{q,n_q}\} \in \mathcal{D}$ to be ranked. Each pair of query-item $(q, d_{q,i})$, $1 \leq i \leq n_q$ is described by the same set of categorical and numerical features, and assigned a score $y_{q,i}$ that quantifies the relevance of $d_{q,i}$ to q . The objective of LETOR is to infer a ranking model that can accurately assess the relevance of any given query-item pair.

Model Architecture

As discussed in the introduction, we propose to cast LETOR as a DG problem, and view every query as an individual domain. All of the domains/queries share the same learning task and the same input feature space, but have different distributions. To better account for such distributional differences, we parametrize the desired ranking model as a neural architecture that consists of five main components, as illustrated in Figure 1:

- An **item encoder** that transforms each query-item pair $(q, d_{q,i})$ into an encoding vector $\mathbf{h}_{q,i}$;
- An exchangeable **item pooling layer** that collapses the encoding vectors associated with each query q to a single query embedding vector $\mathbf{c}_q$, which is then combined with $\mathbf{h}_{q,i}$ to construct a refined item encoding vector $\tilde{\mathbf{h}}_{q,i}$;
- A parameter-free **query normalization layer** that reduces the distributional differences between item encodings from different queries;
- A **ranking layer** that computes the ranking score using item representations learned from the previous layers;
- An **objective function** that simultaneously promotes the improvement of ranking accuracy and the semantic alignment of feature distributions among queries.

Item Encoder

The item encoder that we consider in this paper is similar to previous works (??; ?). It accepts the features of query-item pairs as input. Given the sensitivity of neural networks to input scaling (??), numerical features are normalized to the interval $[0, 1]$, and categorical features are mapped to dense vectors via embeddings, which are learned jointly with all other model parameters through back-propagation.

Additionally, sometimes the input data may be supplemented with certain order information, such as the predictive results given by an initial LETOR algorithm (?). Different from previous works that model such information using RNNs, we simply incorporate it as an additional numerical feature. While more sophisticated schemes for encoding order information (??; ?) are readily applicable, we found that such a simple option already performs well on the datasets that we tested.

For each pair of $(q, d_{q,i})$, all of the pre-processed features mentioned above are concatenated into an array $\mathbf{x}_{q,i}$. As

noted in (?), direct usage of this feature representation may fail to fully leverage the expressive power of neural models. We thus follow (?), and pass $\mathbf{x}_{q,i}$ through 2 layers of fully connected exponential linear units (ELUs) (?):

$$\begin{aligned} \mathbf{x}_{q,i}^{(1)} &= \text{ELU}(\mathbf{W}^{(1)}\mathbf{x}_{q,i} + \mathbf{b}^{(1)}), \\ \mathbf{x}_{q,i}^{(2)} &= \text{ELU}(\mathbf{W}^{(2)}\mathbf{x}_{q,i}^{(1)} + \mathbf{b}^{(2)}), \end{aligned} \quad (1)$$

the output of which is then concatenated with $\mathbf{x}_{q,i}$ to construct a new feature vector $\mathbf{h}_{q,i}$:

$$\mathbf{h}_{q,i} = \begin{bmatrix} \mathbf{x}_{q,i} \\ \mathbf{x}_{q,i}^{(2)} \end{bmatrix}. \quad (2)$$

Item Pooling Layer

In this work, different queries are treated as different domains, since we observe only a subset of the possible queries/domains during training, additional assumptions are needed to ensure that we can generalize to a new query/domain during testing. A useful technique in the DG literature to generalize to new domains is *domain embedding* (?), which maps domains into the same semantic space. Such a technique captures the domain variations via continuous latent features, and thereby allows effective knowledge transfer between domains.

Along the same line, previous listwise context modeling works (??; ?) choose to construct ‘context encoding’ to capture contextual information using RNNs. Concretely, given the set of feature vectors $\{\mathbf{h}_{q,i}\}_{1 \leq i \leq n_q}$ for query q , these works firstly feed the vectors sequentially into a RNN, and then create the context encoding $\mathbf{c}_q$ by using the final hidden state of the RNN. As $\mathbf{c}_q$ is then used to query the item vectors to generate the final ranking order, it plays a pivotal role in these models. However, the recurrent nature of RNN means that the creating process of $\mathbf{c}_q$ has no direct access to the candidate items except for the last one, and all relevant information has to be propagated by the RNN through the ordered items in a one-by-one manner, which imposes additional memorization burden on the RNN and may incur unnecessary information loss.

Based on the above considerations, in the paper we adopt the self-attention mechanism (??; ?) to create $\mathbf{c}_q$. Concretely, we first feed $\{\mathbf{h}_{q,i}\}_{1 \leq i \leq n_q}$ through a multilayer perceptron MLP_{att} and then a softmax function to generate the attention distribution over the items of the list:

$$a_{q,i} = \frac{\exp(\text{MLP}_{\text{att}}(\mathbf{h}_{q,i}))}{\sum_{k=1}^{n_q} \exp(\text{MLP}_{\text{att}}(\mathbf{h}_{q,k}))}, 1 \leq i \leq n_q. \quad (3)$$

The generated positive weight $a_{q,i}$ in (3) can be interpreted as an estimation of the probability that item $d_{q,i}$ is the right place to focus on for downstream finer-grained ranking. Based on the attention distribution, we calculate $\mathbf{c}_q$ as the attention-weighted mean of the item vectors:

$$\mathbf{c}_q = \sum_{i=1}^{n_q} a_{q,i} \mathbf{h}_{q,i}. \quad (4)$$

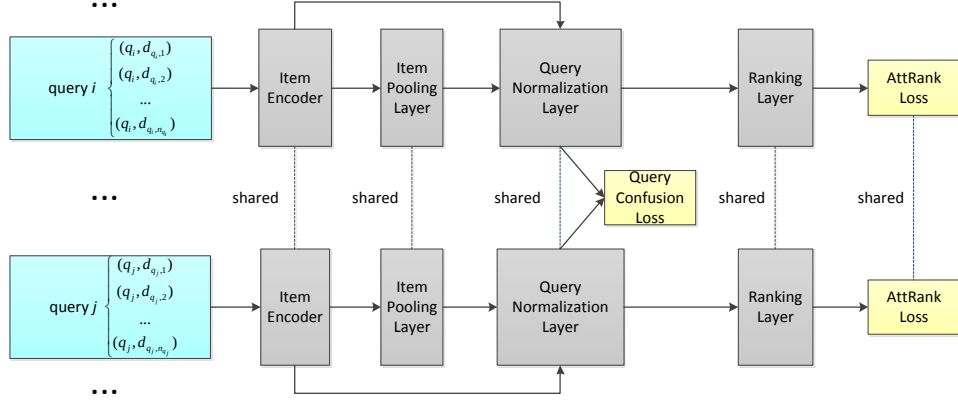


Figure 1: The overall architecture of Query-Invariant Listwise Context Model (QILCM).

Compared to previous models that simply adopt the last hidden state of RNN, attention layer defined in (3) and (4) may extract a more informative global context encoding $\mathbf{c}_q$ since higher weights are assigned to items that are estimated to be more relevant for context modeling. We then combine $\mathbf{c}_q$ with the original item representations to form the refined representation vectors as:

$$\tilde{\mathbf{h}}_{q,i} = \begin{bmatrix} \mathbf{c}_{q,i} \odot \mathbf{h}_{q,i} \\ \mathbf{h}_{q,i} \end{bmatrix}, 1 \leq i \leq n_q, \quad (5)$$

where $\odot$ denotes the Hadamard product. $\tilde{\mathbf{h}}_{q,i}$ is regarded as the refined representation since it encodes both the listwise contextual information and the item information that are relevant to the ranking task.

Remark 1. It is interesting to note that the attention pooling strategy adopted here is a variant of the orderless attention mechanism considered in (?; ?), where all items are independent instead of being sequentially dependent as in RNN-based models. Such a strategy allows the information contained in all item representations to be directly propagated to $\mathbf{c}_q$, which may prevent the long-term memorization problem in RNN-based models.

Query Normalization Layer

As mentioned earlier, the feature distributions of candidate items for different queries are often different, which makes the learning of an effective global ranking function difficult. This problem cannot be completely solved by adopting the context representation presented in the previous section, as the ranking model still needs to learn to disentangle the information about the ‘global’ differences between queries and the ‘local’ differences between items for each query, and then focus on the latter to perform effective ranking.

To address the distributional differences among queries, we borrow ideas from previous DG approaches (?; ?; ?), and encourage the ranking system to learn *query-invariant* latent representations that have similar or even identical distributions across all queries, with the hope that the system may generalize better to unseen queries by eliminating the detrimental influence of inter-query variability.

Additionally, compared with general DG problems, LETOR has its own distinct characteristics that can be exploited to facilitate the learning of query-invariant representations. In particular, the candidate items for each query is available as a whole, which makes the problem of characterizing the underlying distributions easier. For example, for any query q , we can directly compute the attention-weighted per-dimension mean and variance vectors of its associated feature distribution as:

$$\bar{\mathbf{c}}_q = \sum_{i=1}^{n_q} a_{q,i} \tilde{\mathbf{h}}_{q,i}, \quad (6)$$

$$\bar{\sigma}_q^2 = \sum_{i=1}^{n_q} a_{q,i} (\tilde{\mathbf{h}}_{q,i} - \bar{\mathbf{c}}_q)^2. \quad (7)$$

As is the case with the context vector (4), the attention weights (3) are adopted here to emphasize items that are estimated to be more important. Based on (6) and (7), we can construct a query normalization (QN) layer that directly matches these two feature distribution statistics of any given query to zero and unit vectors, respectively:

$$\bar{\mathbf{h}}_{q,i} = (\tilde{\mathbf{h}}_{q,i} - \bar{\mathbf{c}}_q) \odot (\bar{\sigma}_q + \varepsilon)^{-1}, 1 \leq i \leq n_q, \quad (8)$$

where ε is a small positive constant to avoid division by zero.

The QN transform (8) can be viewed as a straightforward extension of the widely used batch normalization (BN) transform (?), where the key difference is that the latter applies the normalization to a batch of training queries instead of to items in each single query.

Ranking Layer

The normalized hidden representations (8) are passed into another MLP with softmax function to infer the final ranking score:

$$s_{q,i} = \frac{\exp(\text{MLP}_{\text{rank}}(\bar{\mathbf{h}}_{q,i}))}{\sum_{i=1}^{n_q} \exp(\text{MLP}_{\text{rank}}(\bar{\mathbf{h}}_{q,i})}), 1 \leq i \leq n_q. \quad (9)$$

Note that (3) and (9) have the same structure, thus the ranking layer is essentially the second attention layer in our model, and $\{s_{q,i}\}_{1 \leq i \leq n_q}$ are the calculated attention weights.

Objective Function

In general, for each training batch $\mathcal{B} \subseteq \mathcal{T}$, QILCM jointly minimizes the *ranking loss* and the *query confusion loss* with a weight parameter λ :

$$\mathcal{L}_{\text{QILCM}} = \mathcal{L}_{\text{rank}} + \lambda \mathcal{L}_{\text{conf}}. \quad (10)$$

In (10), the ranking loss $\mathcal{L}_{\text{rank}}$ can be any suitable loss function that measures the ranking accuracy. In this work, we specifically adopt the AttRank loss proposed in (?) due to its good performance. Let the relevance labels be normalized as:

$$\tilde{y}_{q,i} = \frac{\psi(y_{q,i})}{\sum_{j=1}^{n_q} \psi(y_{q,j})}, 1 \leq i \leq n_q, \quad (11)$$

where $\psi(x)$ is the truncated exponential function that returns $\exp(x)$ if $x > 0$ and 0 otherwise. The ranking loss simply measures the cross entropy between the score (9) and the normalized relevancy labels:

$$\mathcal{L}_{\text{rank}} = \frac{1}{|\mathcal{B}|} \sum_{q \in \mathcal{B}} \left(-\frac{1}{n_q} \sum_{i=1}^{n_q} \tilde{y}_{q,i} \log(s_{q,i}) \right), \quad (12)$$

where $|\cdot|$ denotes the cardinality of a set.

On the other hand, by using the QN layer, the latent feature distribution for any given query is enforced to have zero mean and unit variance in each dimension. However, other types of distributional differences, such as the differences of covariance patterns, can still be present between feature representations from two queries. The query confusion loss is therefore intended to further promote the alignment of distributions of latent features (8) mapped from different queries. In previous DG works, this goal is typically achieved either by minimizing a metric between distributions, or through domain adversarial learning (??) which updates the model parameters to fool a jointly learned domain discriminator that attempts to distinguish between samples from different domains. Due to the large number of domains/queries involved, it is difficult to train a domain discriminator in our setting, and thus we focus on the former approach and choose to minimize the following loss:

$$\mathcal{L}_{\text{conf}} = \frac{1}{|\mathcal{B}|^2} \sum_{q_1 \in \mathcal{B}} \sum_{q_2 \in \mathcal{B}} d_{\text{CH}}(q_1, q_2), \quad (13)$$

where d_{CH} is the Chamfer (pseudo)-distance (CD) (?) for measuring the distance between two point sets:

$$\begin{aligned} d_{\text{CH}}(q_1, q_2) &= \sum_{i=1}^{n_{q_1}} \min_{1 \leq j \leq n_{q_2}} \left(\|\bar{\mathbf{h}}_{q_1,i} - \bar{\mathbf{h}}_{q_2,j}\|_2^2 \right) \\ &+ \sum_{j=1}^{n_{q_2}} \min_{1 \leq i \leq n_{q_1}} \left(\|\bar{\mathbf{h}}_{q_1,i} - \bar{\mathbf{h}}_{q_2,j}\|_2^2 \right) \end{aligned} \quad (14)$$

Remark 2. As both the QN layer and the query confusion regularization are intended to reduce the distributional differences of queries, a naturally arising question is whether these techniques would lead to loss of useful query information and hurt the ranking performance. We will address this issue in the experiments.

Experiments

Baseline Methods

We compare the proposed QILCM with the following three benchmark methods whose implementations are publicly available:

DCM¹ (?) and DLCM² (?) are two recently proposed listwise context modeling methods based on RNNs. Note that the initial formulation of DCM could only handle binary-valued relevancy labels, we resolve this problem by simply replacing the original softmax loss function of DCM with the AttRank loss (9) used in DLCM and our method.

LambdaMART (?) is one of most widely-used listwise LETOR method. We adopt the open-source implementation of this algorithm provided in (?).

Ranking Tasks and Datasets

We evaluate various methods on two ranking tasks which listwise context modeling methods have been successfully applied to:

Ranking Refinement. (?) show that the initial ranked list returned by a LETOR method can be greatly improved by using listwise context modeling methods to rerank the top-ranked results. For this task, we used two large-scale LETOR datasets: Istella-S³ (?) and Microsoft Letor 30K⁴ (?). We followed the experimental protocols described in (?), and adopted LambdaMART to do the initial retrieval, items ranked among the top-100 positions were then re-ranked using various listwise context modeling methods. Note that item lists with less than 100 items were entirely re-ranked in our experiments.

User Choice Ranking. (?) show that listwise context modeling methods can accurately predict the user's choice among alternative commodities. For this task, we used Air-line Itinerary⁵, which is an anonymized version of the dataset used in (?), each record in the data corresponds to a query, and contains the candidate items presented to a customer. The items that the customers purchased are positive samples and others are negative samples.

Statistics of the used datasets are summarized in Table 1.

Evaluation Metrics

For each task, we adopt the evaluation metrics used in prior work. For ranking refinement tasks, we follow (?)

¹<https://www.dropbox.com/s/swghso88q0s3hp7/code.zip>

²<https://github.com/QingyaoAi/Deep-Listwise-Context-Model-for-Ranking-Refinement>

³<http://quickrank.isti.cnr.it/istella-dataset/>

⁴<https://www.microsoft.com/en-us/research/project/mslr/>

⁵https://www.dropbox.com/s/qzyt0xwn4u2a1ed/data_full_anonym.zip

Dataset	Queries	Items	Rel.	Feats.
Airline Itinerary	33951	1,089k	2	17
Microsoft 30k	31531	3,771k	5	136
Istella-S	33018	3,302k	5	220

Table 1: Characteristics of the datasets used in the experiments: number of queries, items, relevance levels, and features.

and use the standard discounted cumulative gain (NDCG) metrics that include NDCG@1, NDCG@3, NDCG@5, and NDCG@10. For Microsoft 30k dataset, an additional metric NDCG@50 was also evaluated. On the other hand, for user choice ranking tasks where relevance feedbacks are binary-valued, we follow (?) and use top precision metrics that include P@1(Precision@1) and P@5.

Model Training

Each dataset is split in train, validation and test sets according to a 60%-20%-20% scheme. The validation set was used to select the optimal hyperparameters for all involved methods. We did not perform extensive hyperparameter search for the proposed model, and used virtually the same architecture throughout all the experiments and datasets. More specifically, the dimensions of the nonlinear transformations (1) in the Input Encoder were fixed as 100, while MLPs used in (3) and (9) consist of 2 hidden layers with either 256 or 128 ELUs. The models were trained with the Adam algorithm (?) with a learning rate of 0.001, batch size of 80. Training generally converged after less than 100 passes through the entire training dataset.

Comparisons with Baseline Methods

The performance comparison results of various methods are reported in Table 2. To eliminate the influence of random initiations, all results are averaged over 20 runs. As is shown in this table, QILCM significantly outperforms all the baselines.

Ablation Studies

To elucidate the contributions of the main components of our system, in this section, we test several variants of the proposed model. The tested implementations include:

- **Variant 1:** Our model with the attention-weighted context encoding (4) replaced by simple average pooling of the item encoding vectors. Accordingly, the attention-weighted statistics (6) and (7) are replaced with standard mean and variance;
- **Variant 2:** Our model without the domain confusion loss;
- **Variant 3:** Our model without the domain confusion loss and QN layer.

The performance results of different implementations of the proposed methods are shown in Table 3, which shows that QILCM consistently achieves the best performance among all model variants.

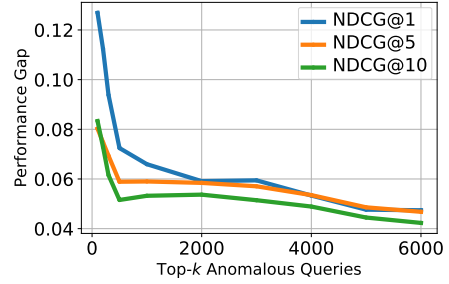


Figure 2: Performance gap between QILCM and the best-performing baseline (DLCM) on Microsoft 30k dataset.

Anomalous Query Analysis

To further investigate the performance of QILCM, in this section we conducted additional analysis of experimental results on the Microsoft 30k dataset. Concretely, we firstly followed (?), and constructed vector representation for each query by averaging over the feature values of the top k ranked items (k was set as 10 during the experiments). After that, we fitted Isolation Forest (?) to the training queries, and then used the learned model to assign a ‘anomaly score’ to each query in the test set. Intuitively, this score quantifies how different a query is from the majority of training data, and we examine the performance difference between QILCM and the best-performing baseline (DLCM) for queries with different levels of anomaly. As shown in Figure 2, the performance gap between QILCM and DLCM is significantly widened for more anomalous queries. For example, the NDCG@1 gap between QILCM and DLCM is increased from 0.047 to 0.121, which clearly demonstrates the advantage of tackling heterogeneous queries using the proposed DG perspective.

Conclusion

In this paper, we introduce a DG formulation of the LETOR problem and propose a novel neural architecture for DG in this LETOR context. We evaluate our techniques on three benchmark datasets, demonstrating that the proposed approach outperforms previous state-of-the-art approaches by a substantial margin. *Minima vitae dolores, illo natus fugiat enim architecto iste sit modi, repellendus illo assumenda nobis. Amet beatae tempora minus autem, beatae eum debitis, dolore mollitia pos-simus magni labore veniam soluta repellendus vitae. Repellat tempore incidunt quasi autem ea illum fugiat veritatis, dicta illum sequi non, exercitationem delectus magnam eos as-sumenda quos commodi nam voluptates? Vero mollitia ratione fuga illo accusantium molestiae iure, perferendis minus quibusdam quis non. Dicta iure odio, inventore at temporibus porro. Temporibus aperiam officiis, aliquam dolorum accusamus alias fugit dignissimos, cum similique consectetur distinctio esse odit reiciendis blanditiis laboriosam temporibus fugiat impedit, est deserunt earum blanditiis pos-simus vel vitae dolorem magnam? Quaeat quis perferendis ipsam repudiandae modi optio exercitationem nesciunt vero eos quasi, facilis perferendis iusto iure dignissimos pos-simus sit eaque reiciendis voluptatibus sequi animi, vel so-*

Dataset	Metrics	QILCM	DCM	DLCM	LambdaMART	Improv.	P-value
Airline Itinerary	P@1	*0.2833	<u>0.2618</u>	0.2562	0.2327	8.21%	2.40×10^{-23}
	P@5	*0.6958	<u>0.6586</u>	<u>0.6651</u>	0.6239	4.61%	1.22×10^{-20}
Microsoft 30k	NDCG@1	*0.5447	0.4938	<u>0.4973</u>	0.4800	9.53%	3.83×10^{-19}
	NDCG@3	*0.5313	<u>0.4827</u>	0.4811	0.4766	10.06%	7.58×10^{-16}
	NDCG@5	*0.5368	<u>0.4904</u>	0.4892	0.4842	9.46%	1.60×10^{-16}
	NDCG@10	*0.5564	0.5093	<u>0.5135</u>	0.5061	8.35%	2.27×10^{-15}
	NDCG@50	*0.6482	0.6127	<u>0.6146</u>	0.6092	5.46%	4.51×10^{-15}
Istella-S	NDCG@1	*0.7023	0.6762	<u>0.6873</u>	0.6644	2.18%	1.74×10^{-10}
	NDCG@3	*0.6696	<u>0.6552</u>	0.6537	0.6378	2.20%	1.99×10^{-8}
	NDCG@5	*0.6953	<u>0.6846</u>	0.6831	0.6741	1.56%	4.25×10^{-10}
	NDCG@10	*0.7645	0.7558	<u>0.7566</u>	0.7456	1.04%	4.67×10^{-7}

Table 2: Performance comparison of various methods. The results are averaged over 20 random runs, and the best ones are marked with *. The last two columns show the improvement of QILCM over the best baseline algorithm (highlighted with underline), and the corresponding Student’s t-test P-values.

Dataset	Metrics	QILCM	Variant 1	Variant 2	Variant 3
Airline Itinerary	P@1	*0.2833	0.2749	0.2762	0.2694
	P@5	*0.6958	0.6613	0.6803	0.6724
Microsoft 30k	NDCG@1	*0.5447	0.5287	0.5359	0.5139
	NDCG@3	*0.5313	0.5127	0.5294	0.4970
	NDCG@5	*0.5368	0.5230	0.5347	0.5030
	NDCG@10	*0.5564	0.5459	0.5538	0.5229
	NDCG@50	*0.6482	0.6363	0.6438	0.6331
Istella-S	NDCG@1	*0.7023	0.6966	0.6982	0.6837
	NDCG@3	*0.6696	0.6654	0.6672	0.6628
	NDCG@5	*0.6953	0.6936	0.6931	0.6923
	NDCG@10	*0.7645	0.7622	0.7637	0.7602

Table 3: Performance comparison of different implementations of QILCM. The results are averaged over 20 random runs, and the best ones are marked with *.

luta vitae, accusamus quas est distinctio tempore aliquam natus adipisci rerum provident dolore. Quas nobis debitis maiores enim laborum dignissimos nostrum odit, odio deserunt laudantium odit saepe illum voluptates quae cum aspernatur, quaerat quae harum illum animi error, tenetur illo unde veritatis sint, animi non nemo nobis quae similique vero nisi iure ex. Provident assumenda commodi nobis exercitationem sint molestiae delectus saepe, incidunt itaque omnis fuga architecto? Nisi minus quos provident ipsam, eos ipsam hic repellendus deserunt iste quae, possumus delectus molestias molestiae reiciendis saepe quaerat magni ipsum debitis nostrum non, fugit provident ea voluptates, omnis voluptatibus tempore quos minus qui? Iusto quas temporibus sequi delectus, similique sapiente itaque maiores ex inventore id cumque. Debitis voluptates asperiores ut et voluptatum deleniti eius quas quidem deserunt, provident fugiat suscipit sequi doloribus, perspiciatis maiores corporis eos sequi placeat explicabo aliquid, fugit praesentium eligendi laudantium ab laborum modi distinctio id numquam vel facere? Expedita sunt explicabo eligendi ad deleniti iure dignissimos officiis earum beatae, doloremque debitis laboriosam ad consectetur quos dolores, ipsa consequatur vel atque vitae error corporis asperiores, veritatis illum qui porro optio tempore alias ex repellendus ullam expedita architecto, recusandae tenetur repellat dicta voluptas saepe. Natus libero illum, eveniet mollitia expedita minima hic laudantium laboriosam porro fugiat, ipsam autem nisi atque ipsa

magni debitis, obcaecati dolorem aperiam sequi facilis temporibus voluptatibus sunt. Illo suscipit accusantium eligendi sint obcaecati deleniti voluptate maxime animi, sint in preferendis adipisci esse rem eius earum, repellat totam molestiae aperiam qui consectetur? Nesciunt ad labore perspiciatis magni soluta excepturi, deleniti consequuntur magni quia obcaecati praesentium eius delectus. Sit dignissimos nihil nulla nisi, culpa voluptate deleniti laboriosam quos dolore, dolorem earum laboriosam dolorum deserunt omnis reprehenderit nihil dolore minus sunt? Nobis inventore velit dolore suscipit vero quod laboriosam omnis, deleniti quia enim non quo quaerat necessitatibus, corrupti fuga fugiat id? Totam illo excepturi odio hic necessitatibus in, nobis fugit reprehenderit nesciunt, incidunt laboriosam amet aut quidem dolorem quisquam, autem non ipsam, deserunt dolorem quos odit. Quos tempore voluptate culpa, fugit quo consequuntur expedita mollitia magnam repellendus laborum dolorum, reiciendis possumus autem quam laudantium aliquid, nulla totam ad dicta delectus deleniti optio culpa molestias voluptatibus odio. Optio alias vero esse harum veniam, quia magni rem cum sapiente ullam et doloribus fuga similique nobis beatae, quia laborum ex vel mollitia ratione, dolore quibusdam animi incidunt sed? Rem aperiam aspernatur magni odit quae harum, tempore aspernatur pariat. Ex consequatur vero fugit pariat quia deserunt ut voluptates adipisci, exercitationem accusamus quae impedit repudiandae?